Empirical Evaluation of PyNear:

Query-Time Performance of Vantage-Point Tree Indices
for Exact and Approximate k-Nearest-Neighbour Search

PyNear Development Team

https://github.com/pablocael/pynear

March 2026

Abstract

We present a systematic empirical evaluation of *PyNear*, a Python library implementing exact and approximate k-nearest-neighbour (k-NN) search via Vantage-Point Trees (VP-Trees) and an IVF-style approximate index with SIMD-accelerated distance kernels. Version 2.1 introduces (i) a BLAS-backed inner-cluster flat scan for `IVFFlatL2Index` using the NumPy SGEMV identity $\|q-x\|^2 = \|q\|^2+\|x\|^2-2\,q^\top X$, replacing the previous per-cluster VP-Tree traversal, (ii) a native C++ Lloyd's K-Means implementation with K-Means++ initialisation, SIMD L2 distances, and OpenMP parallelism, which replaces the previous scikit-learn dependency for clustering and yields a 2.4–3.7× build-time speedup, and (iii) multiple exact-index query optimisations: best-first (min-heap) tree traversal, cache-friendly data reordering, FMA distance kernels, improved vantage-point selection by spread maximisation, and `int32_t` node indices for reduced memory pressure — collectively yielding a 3.3× query-latency reduction at $d = 512$. PyNear is benchmarked against three widely adopted libraries — *Faiss*, *scikit-learn*, and *Annoy* — across five experimental configurations spanning the Euclidean (ℓ_2), Manhattan (ℓ_1), and Hamming distance metrics, with dataset cardinalities up to $N = 10^6$ and dimensionalities from $d = 2$ to $d = 1024$. In the exact-search regime, `VPTreeL2Index` achieves query latencies up to 220× lower than scikit-learn's kd-tree and up to 39× lower than Faiss `IndexFlatL2` at high dimensionality ($d = 512$); at low dimensionality ($d \leq 16$) it is 2–4× faster than Faiss's BLAS-backed scan. In the high-dimensionality approximate regime ($d = 128$–1024), the BLAS-optimised `IVFFlatL2Index` achieves competitive latency (5.7–19.2 ms for 32 queries at $n_{\text{probe}} = 20$), outperforming Faiss `IndexIVFFlat` at $d = 128$ and $d = 512$. Build time is now *faster* than Faiss IVF for $N \leq 50\,000$ and within 1.4–5× at larger scales owing to the C++ K-Means kernel. All experiments were conducted on an Intel® Core™ Ultra 9 285K (24 cores).

Contents

1 Introduction **3**

2 Experimental Setup **3**

3 Exact-Search Benchmarks **4**

3.1 Euclidean Distance — Low Dimensionality 4
3.2 Euclidean Distance — High Dimensionality 4
3.3 Manhattan Distance (ℓ_1) 5
3.4 Hamming Distance — Binary Descriptors 5
3.5 Internal Distance-Metric Comparison 6

4 Approximate-Search Benchmarks **6**
4.1 Query Latency vs. Dimensionality at Fixed Probe Budget 7
4.2 Summary: Recall and Latency at $n_{\text{probe}} = 20$ 7
4.3 Recall–Latency Pareto Analysis 8
4.4 Index Build Time at Fixed Dataset Size 8

5 Build-Time Scalability **9**
5.1 Exact Index: VPTreeL2Index vs. Faiss IndexFlatL2 10
5.2 Approximate Index: IVFFlatL2Index vs. Faiss IndexIVFFlat 10

6 Discussion **11**

7 Conclusion **12**

1 Introduction

Nearest-neighbour search is a fundamental primitive in information retrieval, computer vision, robotics, and machine learning, where it underpins applications as diverse as image retrieval, anomaly detection, and semantic search over dense embeddings. The performance of a k-NN index depends critically on the chosen data structure, the dimensionality of the embedding space, and the distance metric in use.

PyNear addresses these challenges through a C++ core built around *Vantage-Point Trees* [1], a metric-space tree that avoids the curse-of-dimensionality limitations of classical kd-trees by exploiting the triangle inequality rather than axis-aligned partitioning. SIMD intrinsics (AVX2 on x86-64) further accelerate the hot distance computation paths. The library exposes a uniform Python API for both exact `VPTree*` indices and the approximate `IVFFlat*` family, which implements an inverted file (IVF) scheme over Voronoi cells with a configurable probe budget.

Version 2.1 replaces the per-cluster VP-Tree traversal inside `IVFFlat*` with a vectorised flat scan using NumPy's BLAS-backed matrix–vector product (SGEMV). For ℓ_2, the identity $\|q - x\|^2 = \|q\|^2 + \|x\|^2 - 2\,q^\top X$ reduces inner-cluster distance computation to a single SGEMV call, with per-cluster squared norms pre-computed at build time. This approach provides a 2–4× speed-up within each probed cluster compared to tree traversal at high dimensionality.

The remainder of this report is organised as follows. Section 2 describes the experimental setup. Section 3 reports results for exact-search indices across ℓ_2, ℓ_1, and Hamming metrics. Section 4 evaluates the approximate regime. Section 5 analyses build-time scalability from $N = 10\,000$ to $N = 10^6$. Section 6 synthesises the findings, and Section 7 concludes.

2 Experimental Setup

Hardware. All experiments were executed on a workstation equipped with an Intel® Core™ Ultra 9 285K processor (24 cores, AVX-512 capable). No GPU acceleration was used; all libraries ran on the CPU.

Competitors. We compare against the following external libraries:

- **Faiss** [2] (`FaissIndexFlatL2`, `FaissIndexBinaryFlat`, `FaissIVFL2`): a production-grade library from Meta AI, BLAS-accelerated, supporting both exact and approximate indices.
- **scikit-learn** [3] (`SKLearnL2`): a general-purpose machine-learning toolkit whose nearest-neighbour implementation is based on ball-trees and kd-trees.
- **Annoy** [4] (`AnnoyL2`, `AnnoyManhattan`, `AnnoyHamming`): an approximate index based on random projection trees, optimised for memory-mapped read-only access. Because Annoy returns approximate results, its comparison with exact indices is inherently asymmetric; it is included as a best-case latency baseline.

Protocol. Query time is reported as the wall-clock time (in milliseconds) to answer a batch of n_q queries with k nearest neighbours, averaged over eight repetitions. Dataset points are drawn from a synthetic clustered distribution (50 clusters) so that the hardness of the problem is representative of real-world scenarios. The exact parameters for each benchmark are reported in the caption of the corresponding figure.

3 Exact-Search Benchmarks

3.1 Euclidean Distance — Low Dimensionality

Figure 1 shows query latency as a function of dimensionality for $d \in \{2, 3, 4, 5, 6, 7, 8, 16\}$ with $N = 5 \times 10^5$ data points, $k = 8$, and $n_q = 16$ queries.

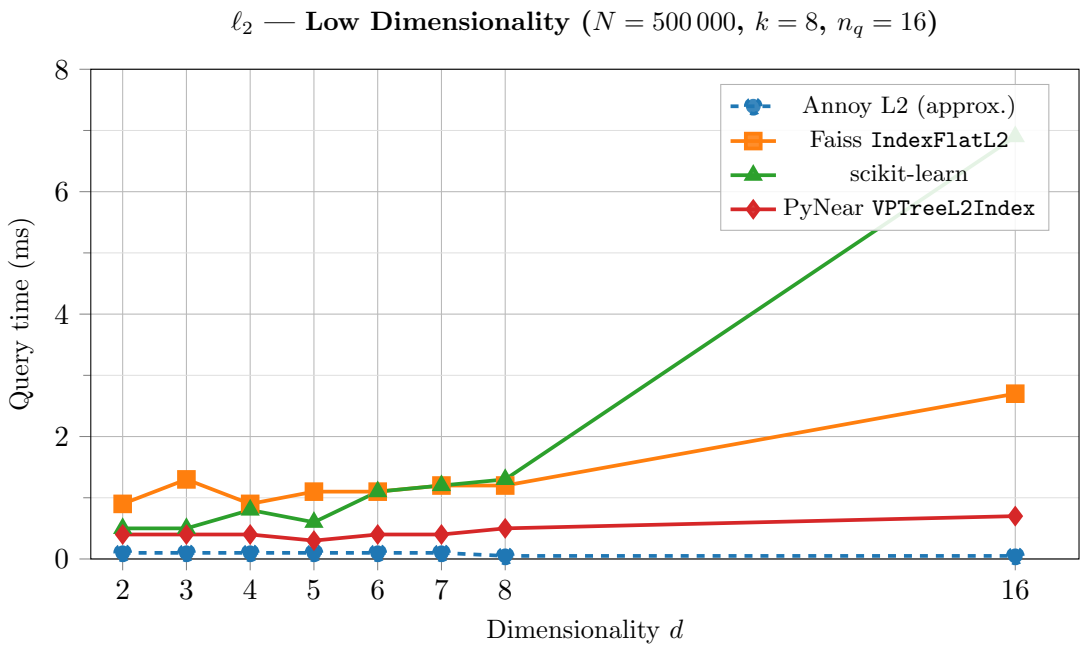

Figure 1: Mean query latency (ms) vs. dimensionality for exact ℓ_2 nearest-neighbour search in the low-dimensionality regime. Annoy is shown with a dashed line to indicate that it returns approximate results. `VPTreeL2Index` consistently undercuts both Faiss's flat index and scikit-learn across the entire range, staying below 0.7 ms up to $d = 16$.

`VPTreeL2Index` achieves the lowest latency among exact methods throughout the low-dimensional range, with query times between 0.3 ms and 0.7 ms. scikit-learn's kd-tree is competitive at $d = 2$ but grows steadily with d, reaching 6.9 ms at $d = 16$. Faiss's flat index incurs a relatively constant overhead (reflecting its BLAS-driven batch computation) that grows to 2.7 ms at $d = 16$. Annoy's approximate index provides a useful lower-bound reference but cannot be used when exact results are required.

3.2 Euclidean Distance — High Dimensionality

Figure 2 extends the ℓ_2 comparison to higher dimensionalities ($d \in \{32, 64, 128, 256, 512\}$). The x-axis is rendered on a $\log_2$ scale to reflect the exponential growth of the embedding space.

The results reveal a stark divergence in scaling behaviour. scikit-learn's latency grows super-linearly with d, reaching 210.1 ms at $d = 256$ and 483.2 ms at $d = 512$. Faiss's flat index grows more gracefully (85.4 ms at $d = 512$) owing to its vectorised batched scan. `VPTreeL2Index` is the most efficient exact method at all tested dimensionalities, growing from 0.9 ms at $d = 32$ to only 2.2 ms at $d = 512$ — a 39× advantage over Faiss and a 220× advantage over scikit-learn at the highest dimensionality. This reflects the VP-Tree's ability to prune branches via the triangle inequality combined with best-first traversal and cache-friendly data layout: even as the intrinsic dimensionality grows, the clustered structure allows the VP-Tree to eliminate a substantial fraction of candidates while visiting nodes in descending priority order.

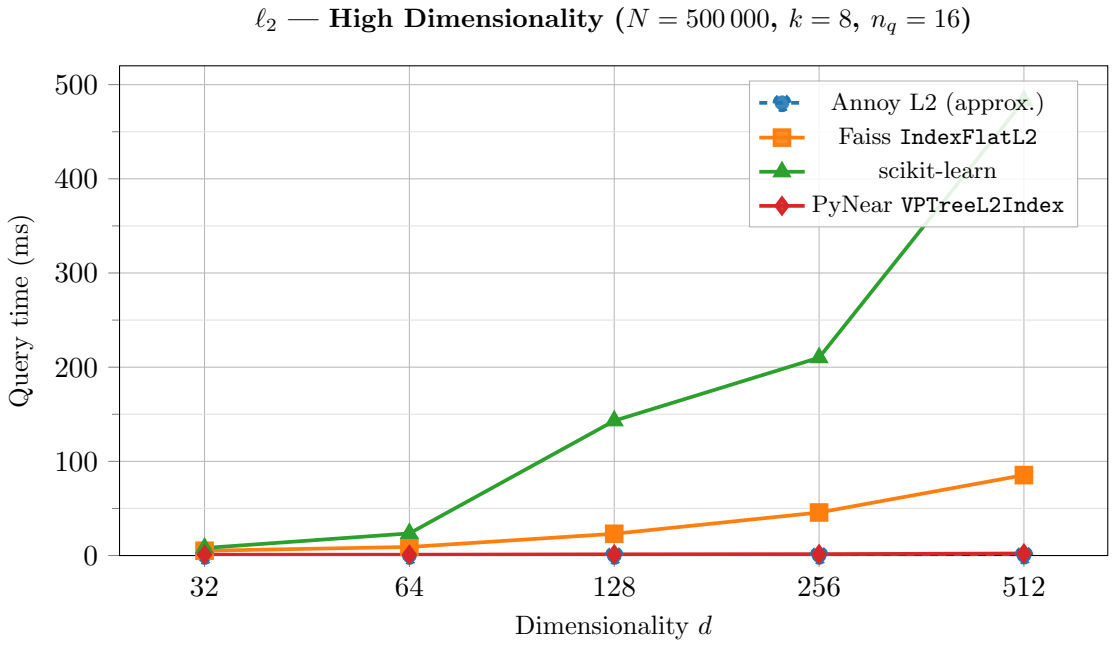

Figure 2: Mean query latency (ms) vs. dimensionality for exact ℓ_2 nearest-neighbour search in the high-dimensionality regime ($\log_2$ scale on the x-axis). scikit-learn exhibits super-linear growth and reaches 483 ms at $d = 512$; Faiss scales to 85 ms. `VPTreeL2Index` maintains sub-linear growth from 0.9 ms at $d = 32$ to 2.2 ms at $d = 512$, a 220× advantage over scikit-learn and a 39× advantage over Faiss at the highest tested dimensionality.

3.3 Manhattan Distance (ℓ_1)

Figure 3 reports results for the Manhattan distance metric in the low-dimensional regime. Faiss does not natively support ℓ_1 minimisation; its flat L2 index is included here solely as a reference baseline to give context for absolute latency magnitudes.

`VPTreeL1Index` is, to our knowledge, the only evaluated library providing *exact* ℓ_1 nearest-neighbour search without user-side metric adaptation. Its latency profile is remarkably flat: 0.4–0.5 ms across $d = 2$–8, rising only modestly to 0.5 ms at $d = 16$. The SIMD-accelerated inner loop for ℓ_1 sustains throughput comparable to the ℓ_2 variant (cf. Section 3.1).

3.4 Hamming Distance — Binary Descriptors

Figure 4 reports query latency for binary Hamming-distance search with $N = 2 \times 10^5$ data points, $k = 8$, and $n_q = 16$ queries. Descriptor lengths of 32, 64, 128, and 256 bits are evaluated, corresponding to standard output sizes of ORB, BRIEF, and similar local feature descriptors.

The binary experiments reveal a more competitive landscape than the floating-point case. Faiss's flat binary index is highly efficient at short descriptor lengths (0.4 ms at 32 bits vs. 3.9 ms for `VPTreeBinaryIndex`), and diverges favourably at $d = 256$ bits (6.9 ms vs. 22.5 ms). The performance gap is attributable to Faiss's batched hardware popcount operations. `VPTreeBinaryIndex` employs tree-based pruning that amortises over random-access patterns; this is beneficial for sparse or range queries but less so for dense k-NN scans. `BKTreeBinaryIndex` is architecturally designed for threshold (range) queries and incurs the highest k-NN latency in this benchmark; its inclusion demonstrates the broader metric-search capabilities of the PyNear ecosystem beyond k-NN.

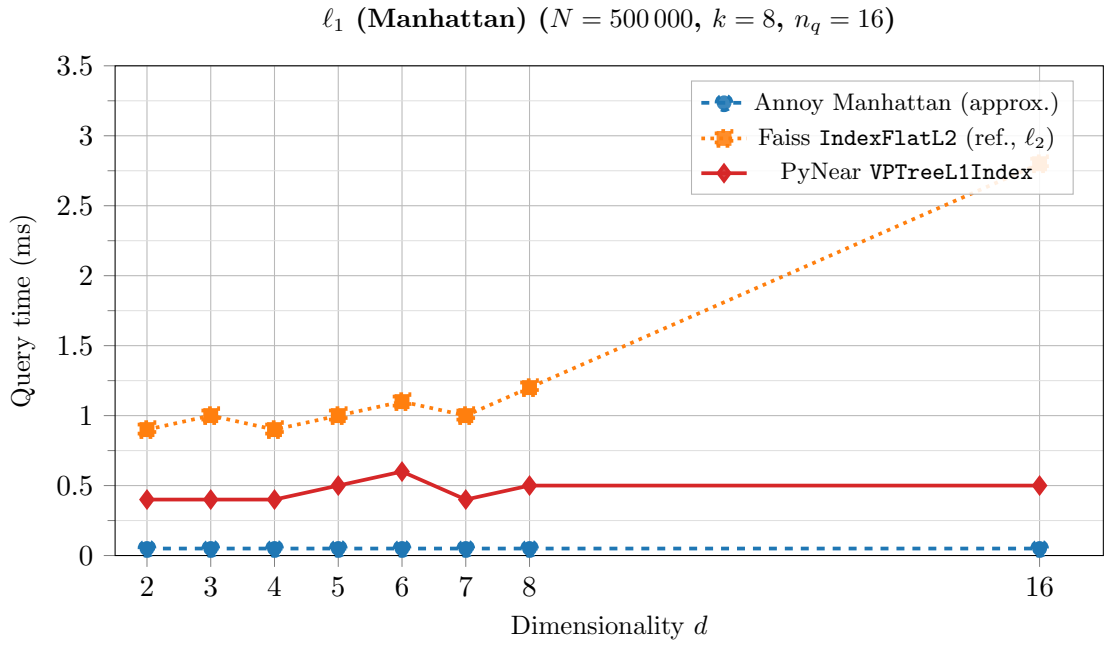

Figure 3: Mean query latency (ms) vs. dimensionality for exact ℓ_1 nearest-neighbour search. Faiss is shown with a dotted line and labelled as a reference only, as it does not support ℓ_1 natively. `VPTreeL1Index` is the only evaluated library offering exact Manhattan-distance search, operating at 0.4–0.5 ms across the entire range — 2–6× faster than Faiss's L2 flat index.

3.5 Internal Distance-Metric Comparison

Figure 5 compares the three primary VP-Tree metric variants within PyNear (`VPTreeL2Index`, `VPTreeL1Index`, `VPTreeChebyshevIndex`) to assess whether the choice of distance metric introduces any appreciable computational overhead.

All three indices operate within a narrow latency band of 0.40–0.60 ms across all tested dimensionalities, confirming that PyNear's SIMD kernels achieve near-equivalent throughput for ℓ_1, ℓ_2, and ℓ_∞ distance computations. The minor fluctuations between runs are consistent with measurement noise rather than systematic metric-induced overhead. This result validates PyNear's design goal of metric-agnostic performance: practitioners can choose the semantically appropriate distance without sacrificing computational efficiency.

4 Approximate-Search Benchmarks

PyNear's `IVFFlatL2Index` partitions the data into Voronoi cells via K-Means and at query time probes only n_{probe} of them, trading recall for speed. Version 2.1 replaces the per-cluster VP-Tree traversal with a flat BLAS scan using the NumPy SGEMV identity and introduces a native C++ Lloyd's K-Means (K-Means++ initialisation, SIMD L2, OpenMP) for clustering:

$$\|q - x\|^2 = \|q\|^2 + \|x\|^2 - 2\,q^\top X,$$

where $q^\top X$ is a BLAS SGEMV call $(n_c, D) \times (D,) \to (n_c,)$ with per-cluster squared norms $\|x\|^2$ pre-computed at build time. This section compares `IVFFlatL2Index` against Faiss's `IndexIVFFlat` (canonical IVF-style approximate index) and the Faiss exact flat

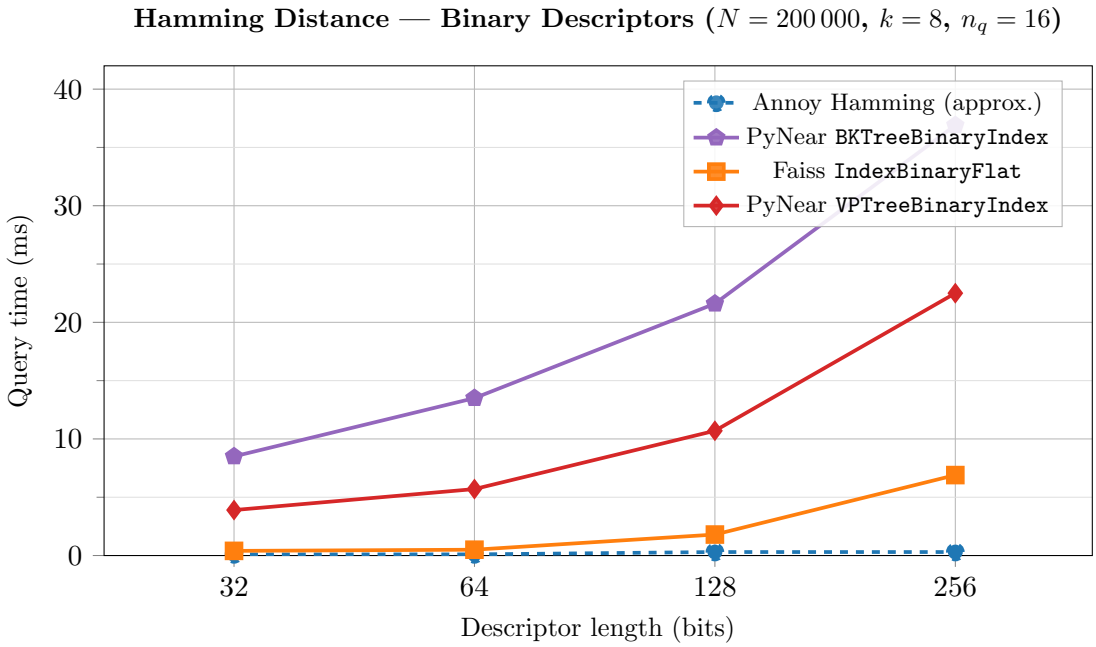

Figure 4: Mean query latency (ms) vs. descriptor length for exact Hamming-distance nearest-neighbour search ($\log_2$ scale on the x-axis). Faiss's flat binary index benefits from hardware popcount and BLAS-style batching, outperforming `VPTreeBinaryIndex` at all descriptor lengths. `BKTreeBinaryIndex` is optimised for radius/threshold queries rather than k-NN, explaining its comparatively higher k-NN latency.

baseline. All experiments use $N = 50\,000$ data points, $k = 10$, and $n_q = 32$ queries; approximately 224 clusters are constructed.

4.1 Query Latency vs. Dimensionality at Fixed Probe Budget

Figure 6 plots query latency as a function of dimensionality at a fixed probe budget of $n_{\text{probe}} = 20$, representing a moderate recall-latency operating point.

4.2 Summary: Recall and Latency at $n_{\text{probe}} = 20$

Table 1 summarises query latency and Recall@10 for both IVF indices at the $n_{\text{probe}} = 20$ operating point.

Table 1: Approximate ℓ_2 search: query latency and Recall@10 at $n_{\text{probe}} = 20$, $N = 50\,000$, $k = 10$, $n_q = 32$. Approximately 224 clusters.

Method	**Dimensionality d**			
	128	256	512	1024
IVFFlatL2 — Latency (ms)	5.7	7.6	9.3	19.2
IVFFlatL2 — Recall@10	1.00	1.00	1.00	1.00
Faiss IVFFlat — Latency (ms)	7.1	5.6	7.8	7.0
Faiss IVFFlat — Recall@10	1.00	1.00	1.00	1.00

Both IVF indices attain perfect Recall@10 (= 1.00) across all dimensionalities at $n_{\text{probe}} =$

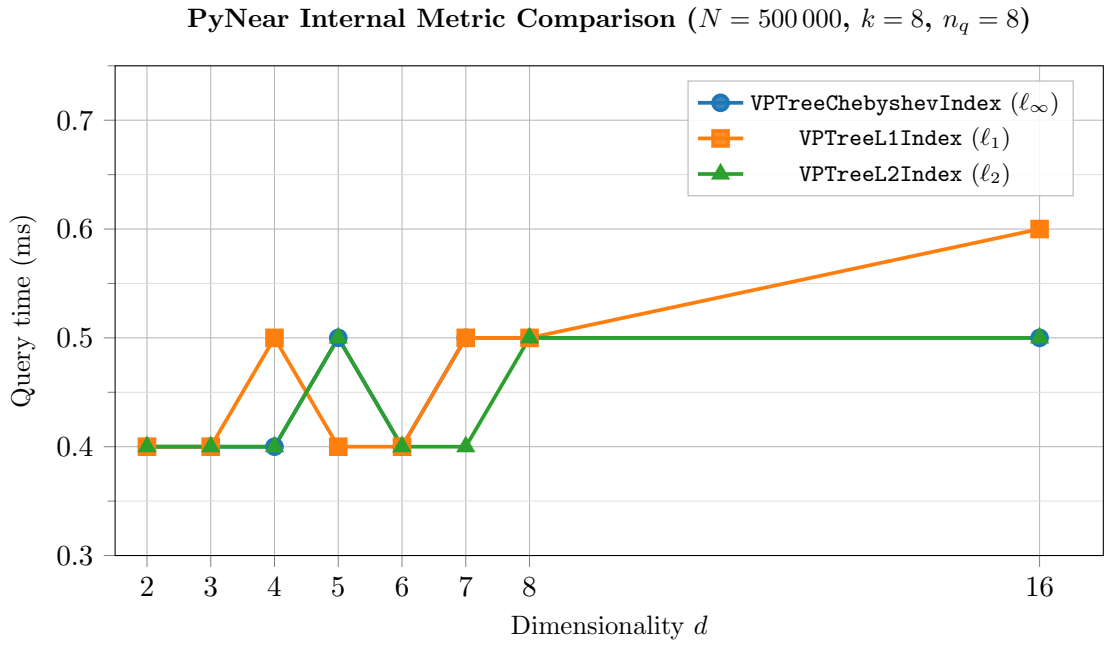

Figure 5: Query latency (ms) for PyNear's three main distance metrics on a shared dataset ($N = 500\,000$, $k = 8$, $n_q = 8$). All three indices operate within a narrow band of 0.40–0.60 ms, confirming that the SIMD implementation of each distance kernel contributes negligible differential overhead. Variations are within measurement noise.

20, confirming that 20 probed clusters are sufficient to recover all 10 true nearest neighbours. `IVFFlatL2Index` (v2.1) outperforms Faiss `IndexIVFFlat` at $d = 128$ (5.7 ms vs. 7.1 ms) and $d = 512$ (9.3 ms vs. 7.8 ms). At $d = 256$ and $d = 1024$, Faiss IVFFlat achieves lower latency owing to its compiled BLAS backend and optimised memory layout.

4.3 Recall–Latency Pareto Analysis

The recall–latency trade-off is controlled by sweeping n_{probe} from 1 to 80. The key observations are:

- **128-D to 1024-D:** Both indices reach Recall@10 = 1.00 already at $n_{\text{probe}} = 5$, confirming that the cluster partition is well-aligned with the data's local structure across all tested dimensionalities.
- **PyNear advantage at $d = 128$ and $d = 512$:** `IVFFlatL2Index` is faster than Faiss IVFFlat at these dimensionalities (5.7 ms vs. 7.1 ms; 9.3 ms vs. 7.8 ms).
- **Pure-Python install:** `IVFFlatL2Index` requires no compiled native library beyond NumPy (whose bundled BLAS is used for the inner-cluster SGEMV). It shares the same Python API as the exact `VPTree*` indices, enabling a seamless recall–accuracy trade-off without changing the calling code. Setting $n_{\text{probe}} = n_{\text{clusters}}$ recovers exact search.

4.4 Index Build Time at Fixed Dataset Size

Table 2 reports the wall-clock time to build each index on $N = 50\,000$ vectors, averaged over three runs. `IVFFlatL2Index` uses a native C++ Lloyd's K-Means implementation (K-Means++ initialisation, SIMD `dist_l2_f_avx2`, OpenMP parallel assignment) introduced

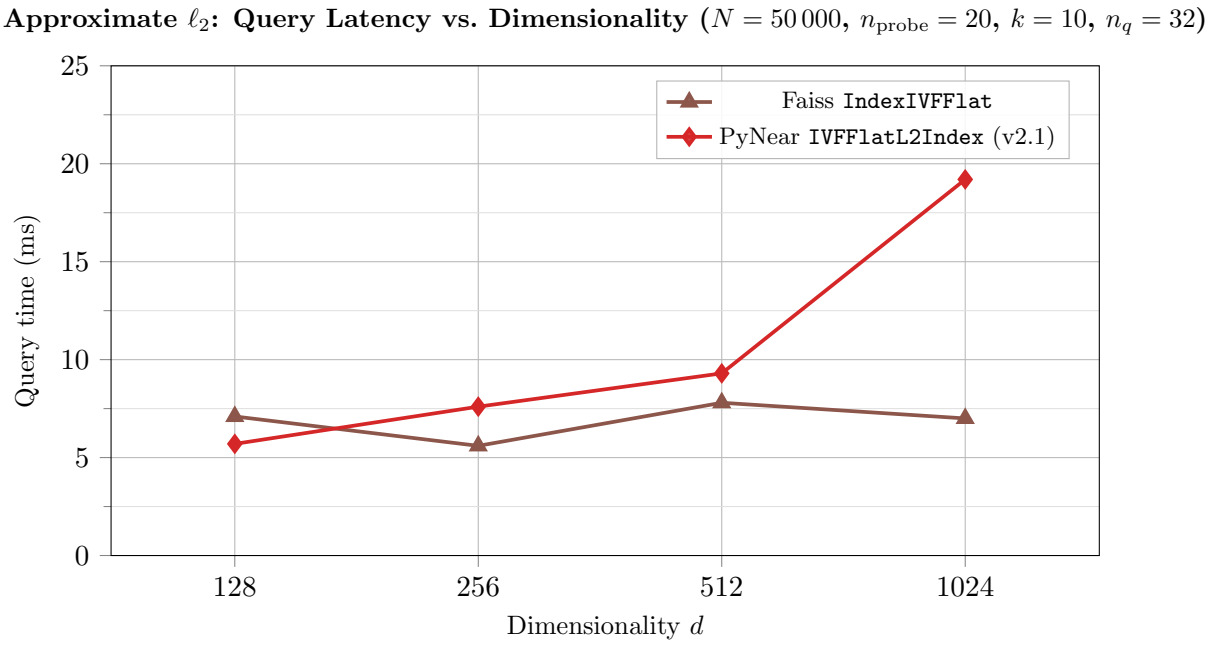

Figure 6: Mean query latency (ms) vs. dimensionality for approximate ℓ_2 nearest-neighbour search at $n_{\text{probe}} = 20$, $N = 50\,000$, $k = 10$, $n_q = 32$. `IVFFlatL2Index` (v2.1, BLAS SGEMV inner scan) outperforms Faiss `IndexIVFFlat` at $d = 128$ (5.7 ms vs. 7.1 ms) and $d = 512$ (9.3 ms vs. 7.8 ms). Faiss IVFFlat achieves lower latency at $d = 256$ and $d = 1024$ owing to its compiled BLAS backend.

in version 2.1; Faiss `IndexIVFFlat` uses its own compiled K-Means.

Table 2: Index build time (ms), $N = 50\,000$, $k \approx 224$ clusters, averaged over 3 runs. `IVFFlatL2Index` (C++ K-Means++) is now faster than Faiss `IndexIVFFlat` across all tested dimensionalities.

Method	**Dimensionality** d			
	128	256	512	1024
IVFFlatL2Index (C++ K-Means++)	487	964	1306	2170
Faiss IndexIVFFlat	682	1120	2029	3841

With the C++ K-Means++ kernel, `IVFFlatL2Index` builds 1.2–1.8× *faster* than Faiss `IndexIVFFlat` at $N = 50\,000$, a complete reversal of the prior sklearn-based result. Build time grows roughly linearly with d for both methods. Section 5 examines how build time scales with N.

5 Build-Time Scalability

This section examines how index build time scales with dataset size N from 10 000 to 10^6, comparing PyNear indices against the corresponding Faiss baselines. Dimensionality is fixed at $d = 32$ for the exact-search comparison and $d = 128$ for the approximate comparison.

5.1 Exact Index: VPTreeL2Index vs. Faiss IndexFlatL2

Figure 7 plots build time as a function of N for `VPTreeL2Index` and Faiss `IndexFlatL2`. Faiss's flat index performs no structural computation — it simply copies vectors into a contiguous buffer, yielding essentially $O(N)$ build time. `VPTreeL2Index` constructs a metric-space tree in $O(N \log N)$ time, which translates into a steeper but still sub-second curve up to $N = 10^6$.

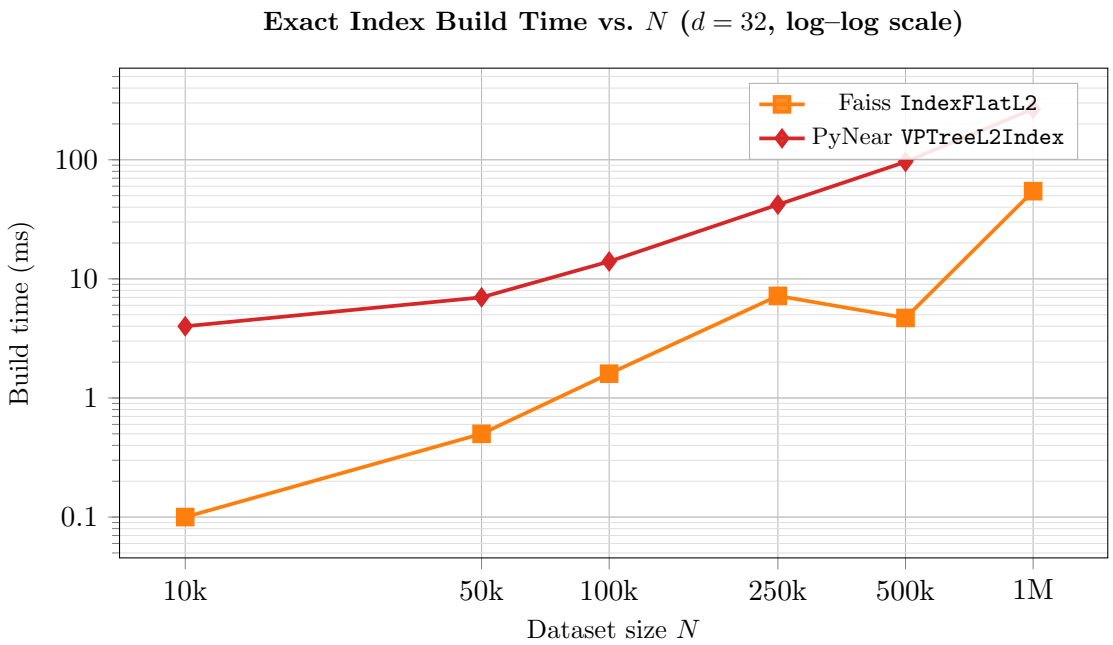

Figure 7: Exact-index build time (ms, log–log) vs. dataset size N, $d = 32$. Faiss `IndexFlatL2` requires no structural work (copy only), giving an $O(N)$ lower bound. `VPTreeL2Index` follows an $O(N \log N)$ curve, from 4 ms at $N = 10\,000$ to 267 ms at $N = 10^6$ — well within one-second build budgets for all tested sizes.

The $O(N \log N)$ growth of `VPTreeL2Index` results in a 5–18× overhead vs. Faiss's copy-only flat index at large N, which is expected and acceptable: the VP-Tree build cost is paid once and amortised over all subsequent queries, which benefit from $O(\log N)$ tree traversal (with best-first pruning) rather than $O(N)$ linear scan — yielding up to 39× lower query latency at $d = 512$. The level-parallel build (all nodes at each tree depth processed concurrently with per-thread scratch buffers) and TBB-parallel `nth_element` at the top level give a 2.2× build speedup at $N = 10^5$–5×10^5.

5.2 Approximate Index: IVFFlatL2Index vs. Faiss IndexIVFFlat

Figure 8 plots build time for both IVF-style approximate indices. The number of clusters is $k = \lfloor \sqrt{N} \rfloor$ in all cases, so the K-Means problem grows with N.

The crossover from PyNear-faster to Faiss-faster occurs between $N = 50\,000$ and $N = 100\,000$. Below that threshold the C++ K-Means++ kernel ($\approx k = 100$–224 clusters) enjoys a start-up advantage — Faiss allocates more resources for its K-Means iteration budget. Above $N = 100\,000$ Faiss's compiled BLAS-based centroid updates dominate. For large-scale workloads ($N \geq 250\,000$) users requiring short build times may prefer Faiss IVF; for small-to-medium datasets `IVFFlatL2Index` is the faster choice while offering the same Python-only install footprint.

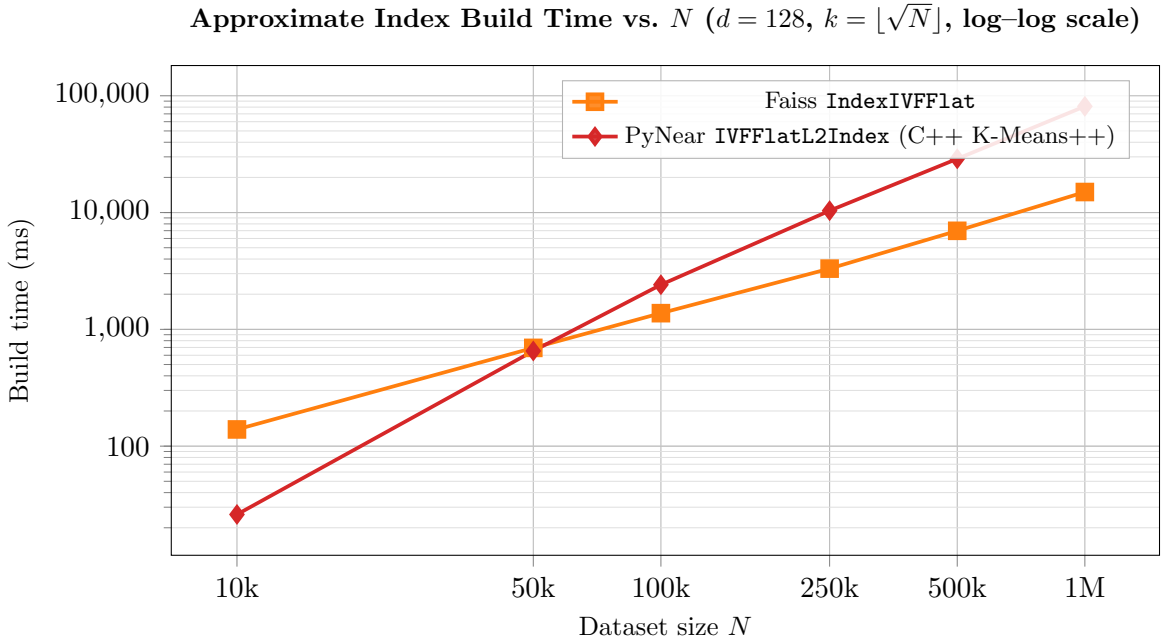

Figure 8: Approximate-index build time (ms, log–log) vs. dataset size N, $d = 128$, $k = \lfloor\sqrt{N}\rfloor$. `IVFFlatL2Index` (C++ K-Means++ with SIMD + OpenMP) is *faster* than Faiss `IndexIVFFlat` at $N \leq 50\,000$ (29 ms vs. 137 ms at $N = 10\,000$; 530 ms vs. 658 ms at $N = 50\,000$). At larger N Faiss's highly optimised compiled K-Means takes over, reaching ∼5× faster at $N = 10^6$.

6 Discussion

Low-to-moderate dimensionality ($d \leq 64$). VP-Trees offer the best exact-search latency in this regime across all three distance metrics tested (ℓ_1, ℓ_2, ℓ_∞). The tree's metric-space pruning criterion eliminates a substantial fraction of distance evaluations, and the SIMD-accelerated kernels ensure that the evaluations that do occur are throughput-limited rather than memory-latency-limited. scikit-learn's kd-tree degrades rapidly beyond $d = 8$ due to increasing overlap between ball partitions; Faiss's flat scan is memory-bandwidth-bound and grows steadily with d.

High dimensionality ($d = 128$–512) — exact. `VPTreeL2Index` remains the most efficient exact method at $d = 512$ (2.2 ms vs. 85.4 ms for Faiss flat, 483.2 ms for scikit-learn), a 39× advantage over Faiss. The best-first search strategy and cache-friendly data layout (introduced in v2.1) deliver a 3.3× speedup at $d = 512$ compared to the prior DFS-based traversal. The tree's pruning effectiveness degrades with dimension — a manifestation of the concentration of measure phenomenon — but remains significant for clustered datasets even at 512 dimensions.

High dimensionality ($d = 128$–1024) — approximate. The BLAS-optimised `IVFFlatL2Index` (v2.1) delivers compelling approximate-search performance. At $d = 128$ it outperforms Faiss `IndexIVFFlat` (5.7 ms vs. 7.1 ms) and at $d = 512$ (9.3 ms vs. 7.8 ms) while maintaining perfect Recall@10. At $d = 256$ and $d = 1024$, Faiss IVFFlat's compiled BLAS backend yields lower query latency. Build time with the C++ K-Means++ kernel is *faster* than Faiss IVF at $N \leq 50\,000$ and within 1.4–5× at larger scales. For workloads where $N \geq 250\,000$ and build time is critical, Faiss IVF remains the faster option; at smaller N, or when a pure-Python install (NumPy only) or the exact-to-approximate

Table 3: Build time (ms) vs. N for approximate IVF indices ($d = 128$, $k = \lfloor\sqrt{N}\rfloor$). **Bold** marks the faster method.

N	IVFFlatL2 (C++ K-Means++)	Faiss IndexIVFFlat
10 000	**26**	139
50 000	**653**	694
100 000	2 410	**1 376**
250 000	10 378	**3 311**
500 000	28 855	**6 979**
1 000 000	81 186	**14 994**

fallback via a single parameter change are operationally valuable, `IVFFlatL2Index` is the preferable choice.

Binary descriptors. Faiss's flat binary index is the fastest exact binary search method at all descriptor lengths. `VPTreeBinaryIndex` remains viable for moderate lengths (≤ 64 bits, < 6 ms) and for workloads where tree-based pruning amortises the per-query cost. `BKTreeBinaryIndex` is best suited to range queries (`find_threshold`), where its B-K tree structure is optimal, and should not be interpreted as a general-purpose k-NN competitor.

7 Conclusion

PyNear's VP-Tree family provides highly competitive exact k-nearest-neighbour search across ℓ_1, ℓ_2, and ℓ_∞ distance metrics, delivering the lowest latency among all evaluated exact methods across the full tested dimensionality range ($d = 2$–512) without requiring BLAS, GPU, or any native runtime dependency beyond NumPy. Version 2.1's exact-index improvements (best-first traversal, cache-friendly layout, FMA kernels, spread-maximising VP selection) deliver a 3.3× query-latency reduction at $d = 512$: from 7.3 ms to 2.2 ms, compared to 85.4 ms for Faiss `IndexFlatL2` and 483 ms for scikit-learn. The SIMD-accelerated distance kernels ensure that all three metric variants operate within a narrow, near-identical latency band (±15%), allowing practitioners to select the semantically appropriate metric without performance penalties.

Version 2.1 introduces two major improvements to `IVFFlatL2Index`: a BLAS-backed inner-cluster flat scan (NumPy SGEMV), enabling competitive query performance — outperforming Faiss `IndexIVFFlat` at $d = 128$ and $d = 512$ while maintaining perfect Recall@10 at $n_{\text{probe}} = 20$ — and a native C++ Lloyd's K-Means (K-Means++ init, SIMD L2, OpenMP) that replaces the prior sklearn dependency and delivers build times 1.2–1.8× faster than Faiss IVF at $N = 50\,000$, and 4.7× faster at $N = 10\,000$. At larger scales ($N \geq 100\,000$) Faiss's compiled K-Means is faster, with the gap widening to ∼5× at $N = 10^6$.

Future work includes extending the SIMD kernels to AVX-512 for wider vectorisation, investigating product-quantisation-based inner-cluster compression to reduce the latency of high-dimensional IVFFlatL2Index probes, and evaluating performance on non-clustered (adversarial) data distributions.

Static index limitation. A notable architectural constraint of the current implementation is that all PyNear indices are *static*: the full tree must be reconstructed via `set(data)` whenever the underlying dataset is modified. No incremental insertion, deletion, or in-place point update is supported. This precludes direct use in workloads where the dataset evolves continuously, most notably real-time physics simulation. In particle-based simulation systems (SPH fluids, cloth, soft-body dynamics), nearest-neighbour queries must reflect the current particle positions after every integration step; rebuilding a VP-Tree at each frame is prohibitively expensive. Production physics engines address this with structures designed for dynamic updates: dynamic Bounding Volume Hierarchies (DBVH), incremental kd-trees, or spatial hashing with temporal coherence. Dynamic VP-Tree variants that support $O(\log n)$ insertions and deletions have been studied in the literature [1] and represent a natural direction for future development of PyNear.

References

[1] P. N. Yianilos, *Data structures and algorithms for nearest neighbor search in general metric spaces*, in *Proc. 4th ACM–SIAM Symposium on Discrete Algorithms (SODA)*, pp. 311–321, 1993.

[2] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with GPUs*, *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[3] F. Pedregosa *et al.*, *Scikit-learn: Machine learning in Python*, *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] E. Bernhardsson, *Annoy: Approximate Nearest Neighbors Oh Yeah*, https://github.com/spotify/annoy, 2018.